Kiểm tra khóa 2

Dựng mạng neural và dùng model có sẵn, có giới hạn giờ.

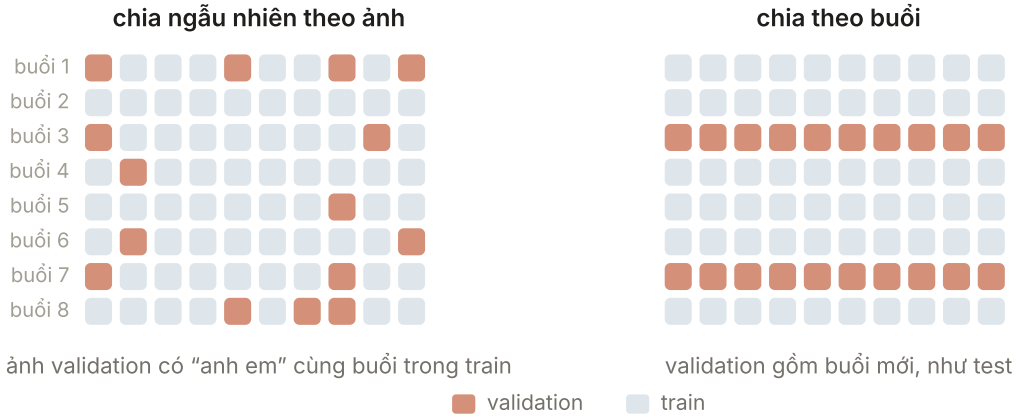
> Khóa 2 nối hai việc: tự dựng và debug mạng neural ở chương 3, rồi dùng lại model đã học ở chương 4. Gặp một bài mới, điểm xuất phát không phải model lớn nhất mà là một pipeline chạy được, đo được và nộp đúng. Buổi này ôn cách ra quyết định và kiểm lỗi theo đúng thứ tự làm một bài thi: đọc đề, debug một batch, chọn cách dùng model pretrained, đọc kết quả, tải lại và ghi file nộp. Bài làm của buổi kiểm tra nằm ở mục bài tập; trang này ôn lại cả quy trình bằng một ví dụ chạy được.

MỤC TIÊU SAU BUỔI NÀY

- Dựng một baseline nhỏ có split, metric và file output đúng định dạng.
- Tìm lỗi shape, loss, gradient và chế độ tầng trên một batch có thể kiểm tay.
- Chọn giữa head-only, fine-tuning, LoRA và zero-shot bằng validation có ý nghĩa.
- Lưu rồi tải lại checkpoint, processor và ánh xạ nhãn để tái tạo dự đoán.

Đi từ đề đến một mốc đo được

Giả sử đề cho ảnh hoạt động của câu lạc bộ, cần xếp mỗi ảnh vào một trong ba nhóm, chấm bằng macro-F1 và nộp một hàng dự đoán cho mỗi ID. Trước hết ghi lại input, output, số lớp, metric và cách chia validation. Ảnh chụp trong cùng một buổi thường giống nhau về ánh sáng, phông nền và người xuất hiện. Nếu test gồm những buổi mới thì validation cũng phải gồm những buổi mới: chia theo buổi. Split ngẫu nhiên theo ảnh cho điểm đẹp hơn nhưng có thể chỉ đo khả năng nhận lại bối cảnh đã thấy.



Hai cách chia cùng 8 buổi ảnh: chia ngẫu nhiên rải ảnh validation vào mọi buổi; chia theo buổi giữ riêng cả buổi, đúng tình huống test là buổi mới.

HỎI NHANH

Ảnh đến từ 40 buổi sinh hoạt, mỗi buổi 30 ảnh, và test là 10 buổi tổ chức sau đó. Chia ngẫu nhiên theo ảnh thì macro-F1 trên validation là 0.93, chia theo buổi thì là 0.71. Con số nào gần với điểm test hơn?

A 0.71, vì validation cũng gồm những buổi chưa thấy

B 0.93, vì validation ngẫu nhiên có đủ mọi buổi

C 0.93, vì hai cách chia dùng chung một kiến trúc model

D 0.82, vì trung bình hai cách chia cho ước lượng ổn hơn

Tiếp theo chạy vài hàng từ đọc input đến file output. Kiểm ID không trùng, không thiếu, đúng thứ tự hoặc đúng giao thức join của đề. Chọn một baseline nhỏ: mạng tự dựng khi đề yêu cầu, feature extraction nếu có encoder phù hợp, hoặc zero-shot khi có asset tương ứng. Baseline phải chạy trong đúng môi trường chấm. Một kiến trúc dựng với trọng số ngẫu nhiên, ví dụ `resnet50(weights=None)`, chỉ là mạng chưa học: nó không mang kiến thức pretrained, dù cùng tên kiến trúc.

HỎI NHANH

Máy thi không có Internet. Code gọi `resnet50(weights=None)`, freeze toàn bộ rồi train head trên 600 ảnh, và macro-F1 validation chỉ được 0.35. Nên nghi gì trước tiên?

A Head quá nhỏ, cần thêm hai lớp ẩn

B Backbone chưa học gì, chỉ là trọng số ngẫu nhiên

C Learning rate của head quá lớn với 600 ảnh

D Macro-F1 luôn thấp hơn accuracy, nên số này bình thường

Debug một batch trước khi tăng ngân sách

Với classifier ba lớp, logits cần shape `(B,3)`, nhãn chỉ số lớp thường có shape `(B,)` và kiểu số nguyên cho `CrossEntropyLoss`. Loss này nhận logits, không cần gọi softmax trước. Kiểm forward cho số hữu hạn, loss có nghĩa và backward tạo gradient ở đúng phần cần học. Thử học thuộc một tập cực nhỏ có nhãn nhất quán giúp kiểm đường training; thành công trên tập nhỏ chưa chứng minh generalization.

HỎI NHANH

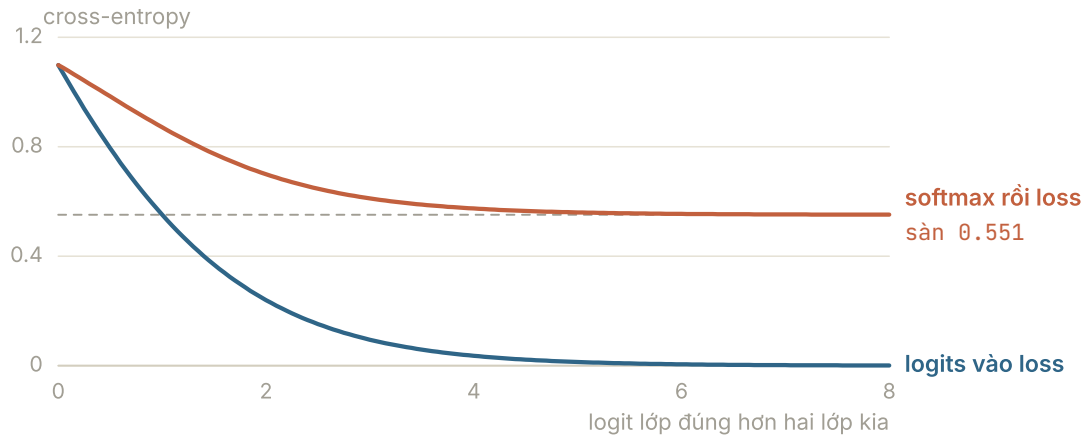
Model ba lớp kết thúc bằng `nn.Softmax(dim=1)`, rồi đưa output vào `CrossEntropyLoss`. Thử cho học thuộc 8 mẫu: accuracy lên 1.0, nhưng loss dừng ở 0.551 và không giảm thêm. Hiểu thế nào?

A Learning rate quá nhỏ, loss chưa kịp về 0

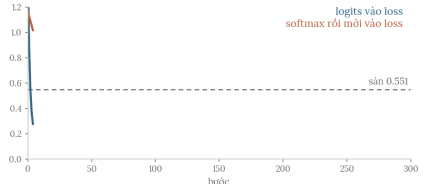
B 8 mẫu quá ít, loss không thể thấp hơn

C Nhãn bị lệch nên loss không xuống thêm

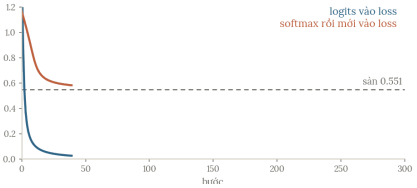
D Softmax bị áp hai lần, nên loss có một mức sàn 0.551



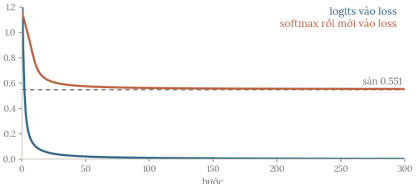
Cross-entropy của một mẫu ba lớp khi logit lớp đúng tăng dần: đưa logits vào loss thì về 0; áp softmax trước thì loss kẹt ở sàn $-\ln(e/(e+2)) \approx 0.551$.



bước 5: loss đúng 0.279, loss khi softmax hai lần 1.019



bước 40: loss đúng 0.028, loss khi softmax hai lần 0.585



bước 300: loss đúng 0.004, loss khi softmax hai lần 0.555

Head của buổi kiểm tra học ba mẫu đầu (mỗi lớp một mẫu) với hai cách tính loss: đưa logits vào thì loss về gần 0; softmax trước thì loss chỉ tiến sát sàn 0.551.

Nếu dùng head-only, so `state_dict` của encoder trước và sau vài bước optimizer: tham số phải giữ nguyên, còn buffer thì theo đúng quyết định của thí nghiệm. Với BatchNorm, freeze weight/bias chưa ngăn running mean đổi khi module ở train mode. Với LoRA, A ngẫu nhiên và B bằng 0 làm gradient của A bằng 0 ở bước đầu, còn gradient của B thường khác 0. Đó là hành vi đúng theo chain rule, không phải lỗi cần sửa.

HỎI NHANH

LoRA một chiều: $y = (w_0 + ba)x$, với $w_0 = 1$ đã freeze, $a = 0.5$, $b = 0$. Mẫu có $x = 1$, target $t = 3$, loss $(y - t)^2$, SGD với learning rate 0.1. Ở bước 1, gradient của a bằng 0. Ở bước 2, gradient của a bằng bao nhiêu?

A 0

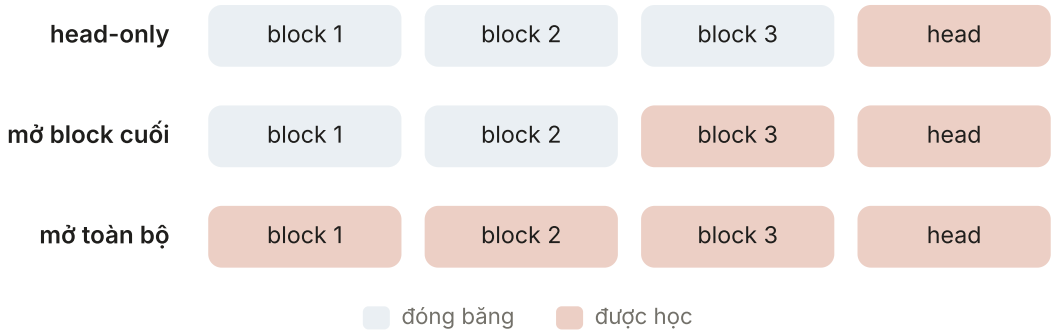
B -2

C -0.76

D -1.9

Chọn bước tiếp bằng một câu hỏi

Head-only hỏi embedding có đọc được nhãn mới không. Fine-tuning hỏi cập nhật phần encoder có cải thiện validation không. LoRA hỏi nhánh sửa ít tham số có đủ chất lượng trong ngân sách không. Zero-shot hỏi kiến thức có sẵn và mô tả lớp có tạo baseline dùng được không. Mỗi thay đổi nên có lý do từ lỗi hoặc giới hạn hiện tại, rồi một phép đo so với mốc cũ trên cùng split.



Ba mức mở tham số, thử theo thứ tự: chỉ head, thêm block cuối, rồi toàn bộ encoder.

HỎI NHANH

Head-only cho macro-F1 0.74 trên train và 0.72 trên validation, và train thêm epoch cũng không đổi gì. Bước tiếp theo hợp lý nhất là gì?

A Train head thêm 50 epoch tới khi train đạt 1.0

B Mở block cuối hoặc thêm LoRA, rồi so trên cùng cách chia

C Đổi sang accuracy, vì macro-F1 khó tăng hơn

D Gộp validation vào train để head có thêm dữ liệu

Không so các mode bằng một cột accuracy nếu metric đề là macro-F1, hoặc chỉ nhìn số tham số để kết luận tổng bộ nhớ. Ghi processor, split, model/revision, seed, batch size, precision, learning rate, phần mở, checkpoint chọn và thời gian thực đo. Giữ một bản có thể nộp trước khi thử một thay đổi tốn nhiều thời gian.

HỎI NHANH

Lần chạy mới vừa mở thêm hai block, vừa tăng learning rate từ 10^{-4} lên $3 \cdot 10^{-4}$, và macro-F1 validation tăng từ 0.72 lên 0.75. Kết luận nào có căn cứ?

A Mở thêm block là nguyên nhân của cải thiện

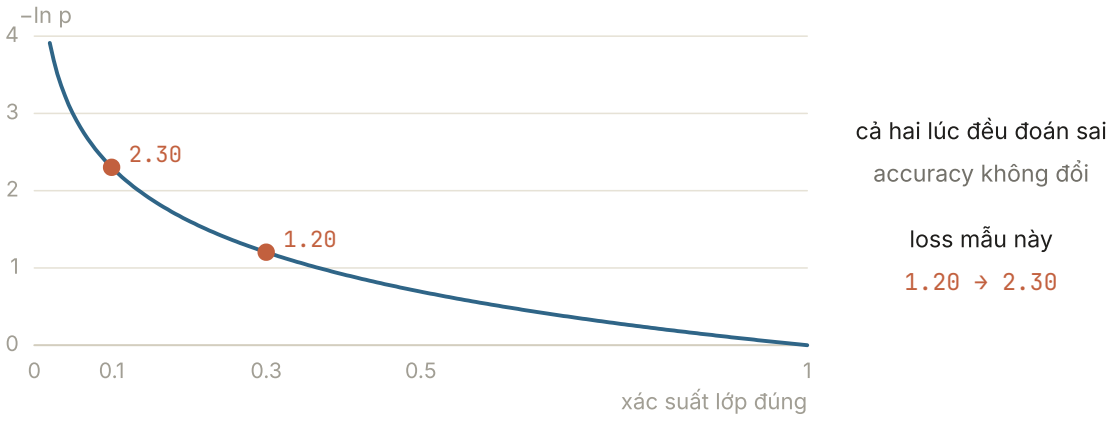
B Learning rate lớn hơn là nguyên nhân của cải thiện

C Hai thay đổi đóng góp như nhau, mỗi cái 0.015

D Cấu hình mới tốt hơn, nhưng chưa tách được nguyên nhân

Đọc kết quả rồi tải lại

Khi train loss tiếp tục giảm nhưng validation loss đạt đáy rồi tăng, ta giữ checkpoint ở đáy theo tiêu chí đã chọn, rồi kiểm lại augmentation, mức mở và regularization. Validation accuracy có thể chưa giảm, vì argmax của từng mẫu chưa đổi; loss thì đã ghi nhận việc model ngày càng tự tin vào những dự đoán sai. Một mẫu sai có xác suất lớp đúng giảm từ 0.3 xuống 0.1 làm loss của riêng nó tăng từ $-\ln 0.3 \approx 1.20$ lên $-\ln 0.1 \approx 2.30$, trong khi accuracy giữ nguyên. Checkpoint được chọn bằng validation đã dùng thông tin validation, nên không gọi điểm ấy là test độc lập.



Một mẫu sai có xác suất lớp đúng giảm từ 0.3 xuống 0.1: argmax vẫn sai nên accuracy giữ nguyên, còn loss của mẫu tăng từ 1.20 lên 2.30.

HỎI NHANH

Từ epoch 3 tới epoch 5, validation accuracy giữ ở 0.84, validation loss tăng từ 0.41 lên 0.58, còn train loss giảm từ 0.20 xuống 0.05. Hiểu thế nào?

A Model ngày càng tự tin vào những mẫu sai, tức đang overfit

B Loss bị tính sai, vì accuracy không đổi thì loss phải đứng yên

C Model chưa học xong, cần train thêm tới khi accuracy tăng

D Validation quá khó, train loss mới là thước đo đúng

Cuối cùng tải checkpoint vào một process mới, nạp processor và bảng nhãn đã lưu, đặt chế độ suy luận, chạy vài mẫu và so với output trước khi lưu. Sau đó dự đoán toàn bộ input cần nộp, kiểm số hàng/ID/nhãn hợp lệ. Một notebook đã chạy trong session cũ có thể vẫn dựa vào biến hoặc cache chưa lưu; bước tải lại giúp phát hiện sự phụ thuộc ấy.

HỎI NHANH

Trong process mới, cho vài ảnh mẫu đi qua processor và checkpoint đã lưu: logits khớp đến từng số với trước khi lưu. Vậy mà file nộp sai nhãn hàng loạt. Code dựng lại `class_names` từ `os.listdir` của thư mục ảnh. Lỗi nằm ở đâu?

A Trọng số nạp sai vì thiếu `strict=True`

B Processor chuẩn hóa ảnh khác lúc train

C Bảng tên lớp dựng từ `os.listdir` bị lệch thứ tự

D Model còn ở chế độ train, nên dropout làm đổi dự đoán

Một pipeline nhỏ có thể đối chiếu

Phần cuối của pipeline (head, metric, lưu/tải, file nộp) có thể kiểm trước khi có encoder thật. Ta thay embedding của ảnh bằng bốn vector hai chiều và đặt trọng số head bằng tay, nên mọi con số đều tính tay được. Lỗi hợp đồng bắt được ở đây không cần đến asset lớn.

Mỗi hàng là một mẫu, mỗi cột là một đặc trưng. Head có ba hàng trọng số tương ứng ba lớp. Với mẫu đầu `(2,0)`, ba logits là `2,0,-2`, nên nhãn dự đoán là 0. Mẫu cuối có nhãn thật 1 nhưng head dự đoán 0: ta cố ý giữ một lỗi để thấy accuracy và macro-F1 không trùng nhau.

Python

```python
import io
import numpy as np
import pandas as pd
import torch
from torch import nn
from sklearn.metrics import accuracy_score, f1_score

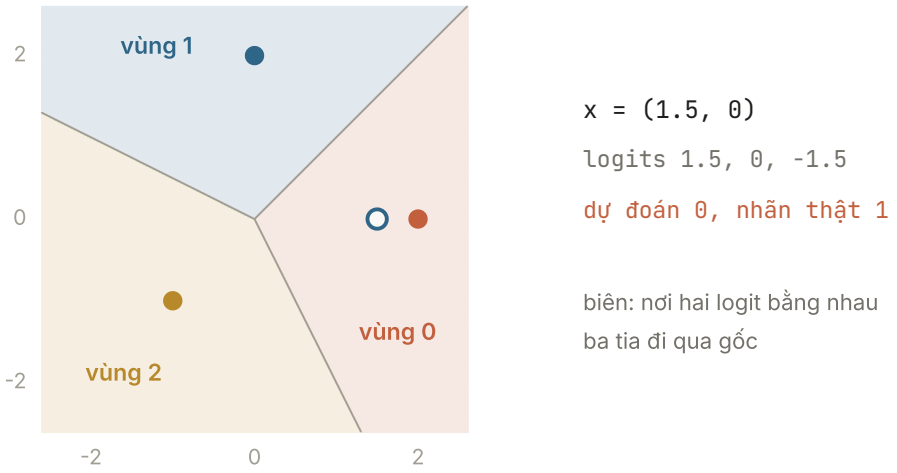
torch.set_num_threads(1)
features = torch.tensor([[2., 0.], [0., 2.], [-1., -1.], [1.5, 0.]])
targets = np.array([0, 1, 2, 1])
class_names = ["nhom_A", "nhom_B", "nhom_C"]
head = nn.Linear(2, 3, bias=False)
with torch.no_grad():
    head.weight.copy_(torch.tensor([[1., 0.], [0., 1.], [-1., -1.]]))
head.eval()
with torch.inference_mode():
    logits = head(features)
    predictions = logits.argmax(dim=1).numpy()
assert logits.shape == (4, 3)
assert np.isfinite(logits.numpy()).all()
print("du doan:", predictions.tolist())
print("accuracy:", accuracy_score(targets, predictions))
print("macro-F1:", round(f1_score(targets, predictions, labels=[0, 1, 2],
                                  average="macro", zero_division=0), 6))
```

KẾT QUẢ

```
du doan: [0, 1, 2, 0]
accuracy: 0.75
```

```
macro-F1: 0.777778
```

Accuracy là $3/4$. Với lớp 0, precision là $1/2$ và recall là 1 nên F1 bằng $2/3$. Lớp 1 có precision 1 và recall $1/2$, cũng bằng $2/3$; lớp 2 có F1 bằng 1. Macro-F1 là trung bình $(2/3 + 2/3 + 1)/3 = 7/9$. Khi metric đề khác accuracy, không thể coi hai con số là cùng một tiêu chí.



Ba vùng quyết định của head đặt tay. Mẫu (1.5, 0) có nhãn 1 (vòng rỗng) nằm trong vùng của lớp 0.

Hình cho thấy vì sao mẫu cuối sai. Logit lớp 0 bằng x_1, lớp 1 bằng x_2, lớp 2 bằng $-x_1 - x_2$; mỗi biên là nơi hai logit bằng nhau, và ba biên là ba tia đi qua gốc. Điểm $(1.5, 0)$ có $x_1 > x_2$ nên rơi vào vùng 0. Muốn sửa, head phải học lại trọng số từ dữ liệu; metric và file nộp chỉ phản ánh đúng những gì head làm.

HỎI NHANH

Cùng target `[0, 1, 2, 1]`. Head A dự đoán `[0, 1, 2, 0]`, head B dự đoán `[0, 1, 1, 1]`. Cả hai có accuracy 0.75. Macro-F1 của head nào cao hơn?

A A, với 0.778, còn B chỉ 0.600

B B, với 0.800, còn A chỉ 0.778

C Bằng nhau, vì accuracy bằng nhau

D B, vì B đoán đúng cả hai mẫu lớp 1

Ta lưu trọng số cùng bảng nhãn và kích thước input, rồi dựng một đối tượng head mới. `BytesIO` đóng vai một tệp trong bộ nhớ, nên ví dụ không ghi gì ra đĩa. Trong bài thi, ghi ra tệp thật và tải lại trong một tiến trình mới như bài 3.4: phép kiểm trong cùng session không bắt được code còn dựa vào biến chưa lưu.

Python

```
buffer = io.BytesIO()
torch.save({"state_dict": head.state_dict(), "class_names": class_names,
            "input_dim": 2}, buffer)
buffer.seek(0)
saved = torch.load(buffer, map_location="cpu", weights_only=True)
restored = nn.Linear(saved["input_dim"], len(saved["class_names"]), bias=False)
restored.load_state_dict(saved["state_dict"])
restored.eval()
with torch.inference_mode():
    restored_logits = restored(features)
assert torch.equal(logits, restored_logits)
sample_ids = [43, 41, 42, 44]
submission = pd.DataFrame({"id": sample_ids, "label": predictions})
csv_buffer = io.StringIO()
submission.to_csv(csv_buffer, index=False)
csv_buffer.seek(0)
read_back = pd.read_csv(csv_buffer)
assert read_back["id"].tolist() == sample_ids
assert read_back["id"].is_unique and len(read_back) == len(features)
assert read_back["label"].between(0, len(class_names) - 1).all()
```

```
print("tai lai trung:", torch.equal(logits, restored_logits))
print(read_back.to_string(index=False))
```

KẾT QUẢ

```
tai lai trung: True
 id  label
 43      0
 41      1
 42      2
 44      0
```

ID 43 vẫn đứng trước 41 vì đó là thứ tự input đã cho. Hợp đồng ở đây là hai cột `id,label`; đề thật có thể yêu cầu xác suất từng lớp, tên cột khác hoặc đúng thứ tự của sample submission. Nếu judge ghép theo thứ tự hàng thay vì theo `id`, một file đúng nội dung nhưng sai thứ tự vẫn gán dự đoán cho nhầm mẫu.

HỎI NHANH

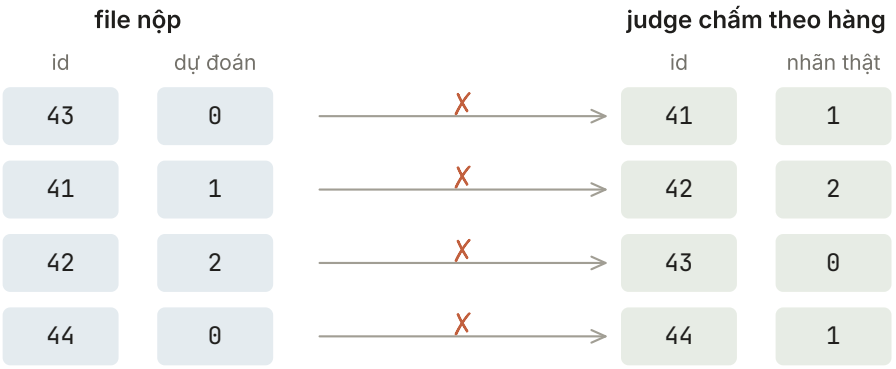
Máy chấm bỏ qua cột `id`, chấm theo thứ tự hàng của file nộp mẫu `41, 42, 43, 44`, với nhãn thật lần lượt là 1, 2, 0, 1. Nộp nguyên file của ví dụ, có thứ tự `43, 41, 42, 44`, thì accuracy bằng bao nhiêu?

A 0.75, vì dự đoán không thay đổi

B 0.50, vì chỉ hai hàng đầu bị đảo

C 0.25, vì hàng ID 44 đúng vị trí

D 0, vì cả bốn hàng đều sai



File nộp giữ thứ tự input 43, 41, 42, 44; judge bỏ cột id và chấm theo thứ tự 41, 42, 43, 44 nên cả bốn hàng đều sai.

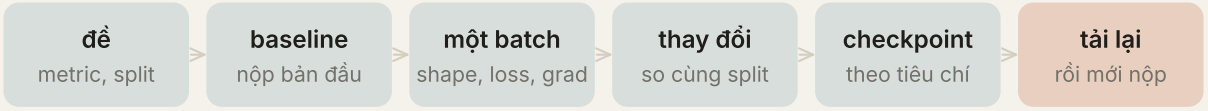
Khi đi thi

- **Đọc metric và đơn vị chia trước khi viết model.** Macro-F1 hay accuracy, ảnh độc lập hay theo buổi: hai câu trả lời này quyết định checkpoint nào được giữ và validation có đáng tin không.
- **Nộp một bản sớm.** Baseline head-only hoặc zero-shot chạy hết đến file nộp hợp lệ; mọi thử nghiệm sau so với mốc này trên cùng split.
- **Kiểm asset trước khi tính giờ train.** Trọng số pretrained phải nạp được từ tệp được cấp, không cần mạng; `weights=None` chỉ là mạng ngẫu nhiên.
- **Debug một batch trước khi chạy dài.** Shape, loss hữu hạn, phần nào có gradient, phần nào nằm trong optimizer, BatchNorm đang ở chế độ nào.
- **Tải lại rồi mới nộp.** Process mới, đọc checkpoint, processor và bảng nhãn đã lưu, so vài logits, rồi assert số hàng, `id` và nhãn hợp lệ theo sample submission.

Tóm tắt

Khóa 2 quy về một chuỗi bằng chứng. Đề được đọc thành input, output, metric và split. Một batch đi qua forward, loss, backward với đúng shape và đúng chế độ. Mỗi cách dùng model pretrained (head-only, fine-tuning, LoRA, zero-shot) trả lời một câu hỏi riêng và

được so trên cùng split. Checkpoint được chọn theo tiêu chí định trước, tải lại trong tiến trình mới và cho ra file nộp đúng hợp đồng.



Quy trình ôn khóa 2: đọc đề, nộp baseline, debug một batch, thử từng thay đổi trên cùng split, chọn checkpoint, tải lại rồi nộp.

Trước buổi kiểm tra, chạy lại ba ví dụ: một mạng nhỏ, một bài feature extraction và một bài LoRA. Với mỗi ví dụ, viết một phép kiểm độc lập từ công thức hoặc dữ liệu nhỏ và giải thích một lỗi có thể xảy ra dù code không báo exception. Khi quên, đọc lại bài 4.1 về hợp đồng asset, 4.2 về pooling và cache, 4.3 về freeze và eval, 4.4 về gradient và rank của LoRA, 4.5 về điểm zero-shot.

Kiểm tra cuối bài

CÂU 1

Validation có 100 ảnh: 80 ảnh nhóm A, 12 ảnh nhóm B, 8 ảnh nhóm C. Một baseline luôn đoán A. Accuracy và macro-F1 của nó bằng bao nhiêu?

A Accuracy 0.80, macro-F1 0.80

B Accuracy 0.80, macro-F1 0.89

C Accuracy 0.80, macro-F1 0.30

D Accuracy 0.33, macro-F1 0.30

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q5.1

CÂU 2

Một neuron $h(x) = \sigma(w_0 + w_1 x)$, với $w_0 = 0$, phân loại dương khi $h(x) > 0.5$. Đổi w_1 từ 1 lên 2. Cái gì thay đổi?

A Vùng dương thu hẹp thành $x > 0.5$

B Vẫn là $x > 0$, nhưng đầu ra xa 0.5 hơn

C Vùng dương mở rộng thành $x > -0.5$

D Không có gì đổi, cả vùng lẫn đầu ra

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q1(b)

CÂU 3

Mạng 10 lớp, mỗi lớp một neuron sigmoid: $o_i = \sigma(w_i o_{i-1} + b_i)$, với $o_0 = x$ và mọi $w_i = 1$. Cận trên của $|\partial o_{10}/\partial w_1|$ và của $|\partial o_{10}/\partial w_{10}|$ là gì?

A Với w_1 là $|x|/4$, với w_{10} là $1/4^{10}$

B Cả hai đều là $|x|/4^{10}$

C Với w_1 là $|x|/40$, với w_{10} là $1/4$

D Với w_1 là $|x|/4^{10}$, với w_{10} là $1/4$

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q4.5

CÂU 4

Early stopping theo macro-F1 validation với patience 2, tức dừng khi 2 epoch liên tiếp không vượt mức tốt nhất. Macro-F1 theo epoch là 0.62, 0.71, 0.69, 0.66, và nếu được chạy tiếp thì epoch 5 sẽ đạt 0.73. Train dừng sau epoch nào, và giữ checkpoint nào?

A Dừng sau epoch 4, giữ epoch 2

B Dừng sau epoch 4, giữ epoch 4

C Dừng sau epoch 3, giữ epoch 3

D Chạy tới epoch 5, giữ epoch 5

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 189

CÂU 5

Trong process mới, nạp checkpoint rồi dự đoán cùng một ảnh hai lần, được hai vector logits khác nhau. Trọng số đã khớp với `state_dict` được lưu. Nguyên nhân khả dĩ nhất là gì?

A Checkpoint bị hỏng khi ghi ra đĩa

B Model còn ở chế độ train, dropout vẫn đang bật

C Chưa bọc `torch.no_grad()` nên forward bị ngẫu nhiên

D Processor tải lại khác phiên bản lúc train

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 124

CÂU 6

Linear attention cần một ánh xạ hữu hạn chiều ϕ sao cho $\phi(q)^\top \phi(k) \approx \exp(q^\top k)$, với hệ số $1/\sqrt{d}$ đã gộp vào q, k. Dùng Taylor bậc hai $\exp(x) \approx 1 + x + x^2/2$ và $(q^\top k)^2 = \sum_{a,b} q_a q_b k_a k_b$, ta lấy $\phi(q)$ gồm số 1, d thành phần q_a, và d^2 tích $q_a q_b/\sqrt{2}$. Với $d = 64$, ϕ có bao nhiêu chiều ℓ, và khi nào chi phí $O(T\ell d)$ thấp hơn $O(T^2 d)$ của attention chuẩn?

A $\ell = 129$, khi $T > 129$

B $\ell = 4161$, với mọi T, vì chi phí tuyến tính theo T

C $\ell = 4161$, khi $T > 4161$

D $\ell = 65$, khi $T > 65$

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 1 · Part 1.1.6

CÂU 7

Adam có hai nhóm tham số: head với learning rate 10^{-3}, block cuối của encoder với 10^{-5}. Ở bước đầu tiên, Adam có sửa độ lệch và ε không đáng kể. Một trọng số của head có gradient 0.002, một trọng số của block có gradient 3.0. Mỗi trọng số thay đổi khoảng bao nhiêu?

A Head khoảng 10^{-3}, block khoảng 10^{-5}

B Head khoảng $2 \cdot 10^{-6}$, block khoảng $3 \cdot 10^{-5}$

C Head khoảng 10^{-3}, block khoảng 10^{-3}

D Head khoảng 10^{-3}, block khoảng $3 \cdot 10^{-5}$

CÂU 8

Encoder 12 lớp, mỗi lớp có hai ma trận chiếu 768×768, là query và value, được gắn LoRA rank 8, không có bias. Tổng số tham số adapter là bao nhiêu?

A 147 456

B 589 824

C 14 155 776

D 294 912

CÂU 9

Zero-shot ghép nhiều template: lớp "xe đạp" lấy trung bình của hai text embedding đã chuẩn hóa, $(0.8, 0.6)$ và $(0.8, -0.6)$. Lớp "ô tô" dùng một template, $(0.6, 0.8)$. Ảnh có embedding $(0.96, 0.28)$, dài 1. Ảnh được gán vào lớp nào nếu quên chuẩn hóa vector trung bình, và nếu chuẩn hóa lại?

A Xe đạp ở cả hai cách, vì lấy trung bình vẫn giữ hướng

B Ô tô khi quên chuẩn hóa, xe đạp khi chuẩn hóa lại

C Xe đạp khi quên chuẩn hóa, ô tô khi chuẩn hóa lại

D Ô tô ở cả hai cách, vì cosine với ô tô là 0.80

CÂU 10

Còn 25 phút. Bản head-only, validation 0.72, đã được nộp. Fine-tune block cuối vừa cho validation 0.76 trong notebook. Lưu, tải lại ở process mới và so vài logits mất 4 phút; dự đoán toàn bộ test mất 6 phút. Nên làm gì?

A Train thêm 15 phút cho validation tăng rồi mới nộp

B Nộp ngay dự đoán từ biến trong notebook cho kịp

C Lưu, tải lại, dự đoán test, kiểm file rồi nộp

D Lấy trung bình logits của hai model rồi nộp luôn

Đọc thêm

- PyTorch — saving and loading models: ôn checkpoint, optimizer state và bước tải lại.
- PyTorch — transfer learning: ôn một vòng train/validation đúng chế độ.
- Recipe thực hành của chương: cách chuẩn bị asset để chạy offline, ghi license và chạy thí nghiệm văn bản, zero-shot với model thật.

Tải gói recipe của chương — gồm các module phụ và hướng dẫn chuẩn bị asset.